
Desenvolvimento de Aplicações Web

Lab 08 - Formulários e Gestão de Utilizadores em Flask

Ano Letivo: 2025/2026

1 Introdução

No desenvolvimento de aplicações com **Flask**, os formulários são frequentemente utilizados para **enviar dados do cliente** (*browser*) ao servidor, onde esses dados podem ser processados, validados e armazenados na base de dados. Esta interação é essencial para funcionalidades como o registo de utilizadores, a autenticação e a edição de informação.

Um dos componentes fundamentais em muitas aplicações é o sistema de autenticação, que permite identificar os utilizadores através de um *login* e controlar o acesso a determinadas funcionalidades da aplicação.

Durante este laboratório, iremos implementar formulários na aplicação desenvolvida até aqui, para permitir a atualização de informações e a autenticação de utilizadores. Este processo ajudará a compreender como os dados são enviados por meio de pedidos HTTP, como podem ser validados no servidor e como podem ser utilizados para gerir sessões de utilizadores numa aplicação web.

2 Objetivos

Neste laboratório iremos:

- Implementar formulários para:
 - criação de novos filmes
 - atualização da informação dos filmes
 - eliminar registos
- Desenvolver e aplicar autenticação de utilizadores
- Aplicar controlo de acessos em algumas páginas

3 Pré-requisitos

- VSCode instalado
- Um browser moderno com ferramentas de desenvolvimento (Chrome, Firefox, Edge, Safari)
- Pasta `PaginaPessoal` dos laboratórios anteriores (Labs 01-07)
- Conhecimentos básicos de HTML, CSS e Python
- Noções gerais de programação (variáveis, condicionais, ciclos)

4 Conceitos-Chave

4.1 Formulários Web

Os formulários web permitem aos utilizadores enviar dados para uma aplicação através de campos como caixas de texto, palavras-passe, botões ou outros elementos. Estes dados são enviados do navegador para o servidor através de pedidos *HTTP*.

Em aplicações desenvolvidas com *Flask*, os formulários são normalmente definidos em *HTML* e processados no servidor utilizando métodos HTTP como *GET* e *POST*.

4.1.1 Métodos HTTP (GET e POST)

Os formulários utilizam métodos HTTP para enviar dados:

- **GET** – utilizado normalmente para obter informação.
- **POST** – usado para casos em que é necessário enviar informação para o servidor, quer seja para guardar a informação ou para validar credenciais de acesso.

O perigo do GET

O método **GET** envia os dados no **URL** do pedido, tornando-os visíveis no histórico do navegador e em logs do servidor. Por esse motivo, **não deve** ser utilizado para **enviar informação sensível**, como palavras-passe ou dados pessoais.

4.1.2 Validação de Dados

Antes de processar ou guardar os dados enviados por um utilizador, é importante garantir que estes são válidos e seguros. A validação pode incluir:

- Verificar se campos obrigatórios estão preenchidos
- Confirmar o formato de um email
- Verificar o tamanho mínimo de uma palavra-passe
- Outras validações

Atualmente, muitas destas validações podem ser realizadas tanto do lado do cliente como do lado do servidor. No entanto, é importante que ambas sejam implementadas, de forma a garantir maior robustez e segurança. Assim, caso uma das validações falhe ou seja contornada, a outra continua a assegurar a verificação dos dados introduzidos.

E as validações em JavaScript?

A validação realizada no lado do cliente **melhora a experiência do utilizador**, permitindo detetar erros rapidamente. No entanto, esta validação **pode ser contornada**, pelo que a validação no lado do **servidor é sempre obrigatória** para garantir a segurança da aplicação.

4.2 Autenticação de Utilizadores

A autenticação é o processo de verificar a identidade de um utilizador. Normalmente envolve:

- Inserção de *username* ou email
- Inserção de palavra-passe
- Verificação das credenciais na base de dados

Se as credenciais estiverem corretas, o utilizador passa a estar autenticado no sistema.

Proteção de palavras-passe

As palavras-passe dos utilizadores **nunca devem ser guardadas em texto simples** numa base de dados. Em vez disso, deve ser utilizado um mecanismo de **hashing**, que transforma a palavra-passe num valor **irreversível**. Desta forma, mesmo que a base de dados seja comprometida, as **palavras-passe reais** dos utilizadores **não são expostas**.

4.2.1 Sessões (Sessions)

Após um *login* bem-sucedido, a aplicação precisa de manter o utilizador autenticado entre diferentes páginas. É precisamente para esse objetivo que são utilizadas as sessões.

Em *Flask*, as sessões permitem guardar informação temporária associada ao utilizador, como por exemplo:

- ID do utilizador autenticado
- Nome do utilizador
- Estado de login

Em *Flask*, as sessões são normalmente implementadas utilizando *cookies* assinados, permitindo guardar informação associada ao utilizador entre diferentes pedidos *HTTP*. Por este motivo, é necessário definir uma `SECRET_KEY` na aplicação para garantir a segurança da sessão.

4.2.2 Logout

O *logout* consiste em terminar a sessão do utilizador, removendo a informação armazenada na sessão. Após o logout, o utilizador deixa de ter acesso a páginas protegidas da aplicação.

4.2.3 Resumo do Fluxo de Autenticação

1. O utilizador preenche um formulário de *login*.
2. O formulário envia os dados para o servidor usando o método *POST*.
3. O servidor valida os dados recebidos.
4. As credenciais são verificadas na base de dados.
5. Se forem válidas, é criada uma sessão de utilizador.
6. O utilizador passa a ter acesso a páginas protegidas.

5 Procedimento

5.1 Formulários

1. Vamos começar por criar um formulário que nos possibilite adicionar um novo filme à nossa BD.
 - (a) Crie um ficheiro `filme_detalhe.html`. Esta página vai ser utilizada para editar e para criar um novo filme.

```
{% extends "layout.html" %}
{% block title %}<h1 class="title">Editar Filme</h1>{% endblock %}
{% block content %}
    <form action="" method="post" name="movie_edit">
        <div class="field">
            <label for="title">Nome</label>
            <div>
                <input type="text" name="title" class="input" required="required"/>
            </div>
        </div>

        <div>
            <label for="year" class="label">Ano</label>
            <div class="control">
                <input type="number" name="year" class="input"
                    min="{{ min_year }}" max="{{ max_year }}" />
            </div>
        </div>

        <div>
            <div>
                <button class="button is-primary is-small">Guardar</button>
            </div>
        </div>
    </form>
{% endblock %}
```

Código 1: Ficheiro `filme_detalhe.html`

Customização do formulário

Pode aplicar os estilos que achar necessários para que o formulário fique bem mais apelativo e incorporado na nossa aplicação.

- (b) Defina no ficheiro `views.py` uma nova função que nos permita adicionar o novo filme:

```
from flask import request, url_for
from models.movie import Movie

def movie_add_page():
    # Verifica se o pedido é do tipo GET
    if request.method == "GET":
        # Cria os valores default vazios para enviar para a página.
        values = {"title": "", "year": ""}
        # Mostra a página com o formulário
        return render_template(
            "filme_detalhe.html", min_year=1887, max_year=
                datetime.now().year, values=values,
        )
    else:
        # Obtém o título enviado no formulário
        form_title = request.form["title"]

        # Obtém o ano enviado no formulário
        form_year = request.form["year"]

        # Cria um objeto Movie com os dados recebidos
        movie = Movie(form_title, year=int(form_year) if
            form_year else None)

        # Obtém a base de dados da configuração da app
        db = current_app.config["db"]

        # Adiciona o filme à base de dados
        movie_key = db.add_movie(movie)

        # Redireciona para a página do filme criado
        return redirect(url_for("filme_page", movie_key=
            movie_key))
```

Código 2: View para adicionar um filme.

- (c) Adicione o seguinte route no ficheiro `server.py`.

```
app.add_url_rule("/new-movie", view_func=views.movie_add_page,
    methods=["GET", "POST"])
```

Código 3: Route para adicionar um filme.

- (d) Coloque um botão na página `filmes.html` para que seja possível navegarmos para a nova página.

```
<a class="botao" href="{{ url_for('movie_add_page') }}">Adicionar
    filme</a>
```

Código 4: Botão para adicionar filme.

- (e) Adicione a função responsável por adicionar um filme na BD.

```
def add_movie(self, movie):
    with dbapi2.connect(self.dbfile) as connection:
        cursor = connection.cursor()
        query = "INSERT INTO MOVIE (TITLE, YEAR) VALUES (?, ?)"
        cursor.execute(query, (movie.title, movie.year))
        connection.commit()
        movie_key = cursor.lastrowid
    return movie_key
```

Código 5: Função de adicionar filme no ficheiro database.py.

- (f) Agora deve ser possível adicionar um novo filme.
2. Tente criar um filme apenas com o título = "" e um ano válido. Este filme foi possível adicionar porque não existem validações do lado do servidor.

- (a) Vamos então criar uma função responsável por validar a informação do nosso formulário.

```
def validate_movie_form(form):
    form.data = {}
    form.errors = {}

    form_title = form.get("title", "").strip()
    if len(form_title) == 0:
        form.errors["title"] = "O nome do filme não pode estar vazio."
    else:
        form.data["title"] = form_title

    form_year = form.get("year")
    if not form_year:
        form.data["year"] = None
    elif not form_year.isdigit():
        form.errors["year"] = "O ano só pode ter números."
    else:
        year = int(form_year)
        if (year < 1887) or (year > datetime.now().year):
            form.errors["year"] = "O ano está fora do intervalo definido."
        else:
            form.data["year"] = year

    return len(form.errors) == 0
```

Código 6: Função de validação do formulário no ficheiro views.py.

- (b) Para que as validações sejam executadas quando o botão de gravar seja carregado, é preciso adicionar o seguinte código logo a seguir ao `else` presente na função `movie_add_page()`.

```
valid = validate_movie_form(request.form)
if not valid:
    return render_template(
        "filme_detalhe.html",
```

```
        min_year=1887,
        max_year=datetime.now().year,
        values=request.form,
    )
```

Código 7: Execução da validação do formulário durante a gravação de informação.

- (c) Volte agora a tentar inserir um filme com o título . Já não deverá ser possível adicionar este, dado que adicionámos a validação do lado do servidor.
3. Vamos agora reutilizar a página *html* criada anteriormente para também editar a informação de filmes já existentes na nossa BD.
- (a) Vamos começar por adicionar uma view para a edição dos objetos. Esta função deverá ser bastante idêntica à função de adição de filmes.

```
def movie_edit_page(movie_key):
    if request.method == "GET":
        db = current_app.config["db"]
        movie = db.get_movie(movie_key)
        if movie is None:
            abort(404)
        values = {"title": movie.title, "year": movie.year}
        return render_template(
            "filme_detalhe.html",
            min_year=1887,
            max_year=datetime.now().year,
            values=values,
        )
    else:
        valid = validate_movie_form(request.form)
        if not valid:
            return render_template(
                "filme_detalhe.html",
                min_year=1887,
                max_year=datetime.now().year,
                values=request.form,
            )
        title = request.form.data["title"]
        year = request.form.data["year"]
        movie = Movie(title, year=year)
        db = current_app.config["db"]
        db.update_movie(movie_key, movie)
        return redirect(url_for("filme_page", movie_key=
            movie_key))
```

Código 8: Função para editar um filme.

- (b) Adicione um botão de Editar na página *filme.html*.

```
<div class="field is-grouped">
    <div class="control">
        <a class="button is-primary is-outlined is-small" href="{{ request.path }}/editar">Editar</a>
    </div>
</div>
```

Código 9: Botão para editar filmes.

- (c) Acrescente a *route* (ficheiro *server.py*) para esta nova funcionalidade e a função responsável por atualizar o objeto na BD (ficheiro *database.py*).
- (d) Se tentarmos de seguida editar um filme, podemos verificar que os valores já introduzidos não aparecem no formulário. Para isso, temos que adicionar os seguintes atributos aos inputs já existentes na página *filme_detalhe.html*:

- `value="{{ values['title'] }}"`
- `value="{{ values['year'] }}"`

4. **Desafio:** Vamos agora criar a opção para eliminar um filme. Para isso precisamos:

- (a) Criar um botão para Apagar na página *filme.html*.
- (b) Adicionar a *route* no ficheiro *server.py*.
- (c) Criar a *view* responsável por eliminar um registo.
- (d) Criar a função responsável por eliminar o objeto na BD.

5. **Desafio Opcional:**

Vantagem do MVC

Graças à utilização do padrão **MVC**, a aplicação não depende diretamente da base de dados utilizada. Se for necessário mudar de tecnologia de base de dados, normalmente basta alterar o código no **Model**, sem modificar as *Views* ou os *Controllers*.

- (a) Faça uma cópia do ficheiro *database.py* e guarde-o como *database_json.py*.
- (b) Modifique as funções deste ficheiro para que os dados dos filmes sejam guardados num ficheiro *JSON* em vez de numa base de dados *SQLite*.
- (c) Implemente pelo menos as seguintes funções:
 - `get_movies()`
 - `get_movie()`
 - `add_movie()`
 - `update_movie()`
 - `delete_movie()`
- (d) No ficheiro *server.py*, altere apenas a inicialização da base de dados para utilizar a nova implementação.
- (e) Execute novamente a aplicação e verifique se todas as funcionalidades continuam a funcionar.

5.2 Gestão de utilizadores (Login)

Até agora qualquer utilizador pode modificar os dados da aplicação. Neste exercício vamos implementar um sistema de **login** para garantir que apenas utilizadores autenticados conseguem alterar a informação da nossa BD.

Vamos criar dois tipos de utilizadores:

- **admin** – pode adicionar, editar e eliminar filmes
- **normaluser** – pode apenas editar filmes

Para implementar esta funcionalidade vamos utilizar a biblioteca **Flask-Login**, que facilita a gestão de sessões de utilizadores numa aplicação Flask.

5.2.1 Instalar dependências

Antes de começar é necessário instalar duas bibliotecas:

- **Flask-Login** – gestão de autenticação
- **passlib** – hashing seguro de palavras-passe

Execute no terminal:

```
pip install flask-login passlib
```

Código 10: Instalação necessária para login em Flask.

5.2.2 Armazenar palavras-passe de forma segura

As palavras-passe **nunca devem ser guardadas em texto simples**. Em vez disso, devemos armazenar apenas um **hash** da palavra-passe. O código abaixo pode ser executado em qualquer ficheiro *python*. A ideia é apenas gerar o *Hash* para as nossas palavras-passe.

```
from passlib.hash import pbkdf2_sha256 as hasher

password = "adminpw"
hashed = hasher.hash(password)

print(hashed)
```

Código 11: Ficheiro *user_config.py* na pasta *configs*.

Mais tarde podemos verificar se a palavra-passe introduzida é correta, utilizando a seguinte função:

```
hasher.verify("adminpw", hashed)
```

5.2.3 Configurar utilizadores

Adicione no ficheiro *settings.py* as seguintes configurações e substitua o hash na password do admin:

```
ADMIN_PASSWORD = "HASH_DA_PASSWORD_ADMIN"

ADMIN_USERNAME = "admin"
```

Código 12: Código de definição de passwords para users.

Palavras-passe seguras

Neste exemplo o utilizador admin é guardado no ficheiro de configuração apenas para simplificar o exercício. Numa aplicação real os utilizadores devem ser guardados numa base de dados.

5.2.4 Criar o modelo de utilizador

Crie um novo ficheiro chamado `user.py`.

```
from flask import current_app
from flask_login import UserMixin

class User(UserMixin):

    def __init__(self, username, password):
        self.username = username
        self.password = password
        self.active = True
        self.is_admin = False

    def get_id(self):
        return self.username

    @property
    def is_active(self):
        return self.active

def get_user(user_id):
    # verificar se é o admin
    if user_id == current_app.config["ADMIN_USERNAME"]:
        return User (
            user_id,
            current_app.config["ADMIN_PASSWORD"]
        )
    # verificar se é um utilizador normal
    db = current_app.config["db"]
    user_data = db.get_user(user_id)

    if user_data:
        return User (
            user_data["username"],
            user_data["password"]
        )

    return None
```

Código 13: Ficheiro `user.py` que representa um utilizador na aplicação.

5.2.5 Configurar o LoginManager

No ficheiro `server.py` adicione o gestor de autenticação.

```
from flask_login import LoginManager
from user import get_user

lm = LoginManager()

@lm.user_loader
def load_user(user_id):
    return get_user(user_id)
```

Código 14: Gestor de autenticação a adicionar no início do ficheiro `server.py`.

Depois, dentro da função `create_app()`, antes da inicialização da BD, inicialize o gestor:

```
lm.init_app(app)
lm.login_view = "login_page"
```

Código 15: Inicialização do gestor de autenticação.

5.2.6 Criar o formulário de login

1. Vamos criar uma nova página *HTML* que deverá reutilizar o mesmo *layout* já criado.

```
{% extends "layout.html" %}
{% block title %}<h1 class="title">Autenticação</h1>{% endblock %}
{% block content %}
<form method="post">
  <div class="field">
    <label class="label">Username</label>
    <div class="control">
      <input class="input" type="text" name="username"
        placeholder="Username">
    </div>
  </div>

  <div class="field">
    <label class="label">Password</label>
    <div class="control">
      <input class="input" type="password" name="password"
        placeholder="Password">
    </div>
  </div>

  <div class="field is-grouped">
    <div class="control">
      <button class="button is-link" type="submit"
        >Login</button>
    </div>
  </div>
{% endblock %}
```

Código 16: Ficheiro `login.html`.

2. É ainda importante adicionar um link no nosso menu que nos possibilite efetuar o *login* ou *logout*.

```
<span class="navbar-item">
  {% if not current_user.is_authenticated %}
    <a class="button is-link" href="{{ url_for('login_page') }}"
      >Log in</a>
  {% else %}
    {{ current_user.username }}
    <a class="button is-link" href="{{ url_for('logout_page') }}"
      >Log out</a>
  {% endif %}
```


Código 17: Link para logins no ficheiro layout.html.

5.2.7 Criar as views de login e logout

No ficheiro `views.py` adicione as *views* de *login* e *logout* e uma função para validar o formulário de *login*:

```
from passlib.hash import pbkdf2_sha256 as hasher
from flask_login import login_required, logout_user, login_user

def validate_login_form(form):
    form.data = {}
    form.errors = {}

    form_username = form.get("username", "").strip()
    if len(form_username) == 0:
        form.errors["username"] = "Username não pode ser vazio."
    else:
        form.data["username"] = form_username

    form_password = form.get("password", "")
    if len(form_password) == 0:
        form.errors["password"] = "Password não pode ser vazia."
    else:
        form.data["password"] = form_password

    return len(form.errors) == 0

def login_page():
    valid = validate_login_form(request.form)
    if not valid:
        return render_template("login.html", form=request.form)
    username = request.form.data["username"]
    user = get_user(username)

    if user is not None:
        password = request.form.data["password"]

        if hasher.verify(password, user.password):
            login_user(user)
            return redirect(url_for("home_page"))

    return render_template("login.html", form=request.form)

def logout_page():
    logout_user()
    return redirect(url_for("home_page"))
```

Código 18: Views de Login e Logout da aplicação.

5.2.8 Adicionar as routes

No ficheiro `server.py` adicione:

```
app.add_url_rule("/login", view_func=views.login_page,
                 methods=["GET", "POST"])

app.add_url_rule("/logout", view_func=views.logout_page)
```

Código 19: Routes de Login e Logout.

5.2.9 Criar tabela de utilizadores na BD

Adicione na base de dados uma nova tabela através da seguinte função no ficheiro *database.py*:

```
# Adicionar este código logo a seguir a "self.create_table()"
self.create_user_table()

def create_user_table(self):
    with dbapi2.connect(self.dbfile) as connection:
        cursor = connection.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS USER (
                ID INTEGER PRIMARY KEY AUTOINCREMENT,
                USERNAME TEXT UNIQUE NOT NULL,
                PASSWORD TEXT NOT NULL
            )
        """)
        connection.commit()
```

Código 20: Função para criar a tabela de User.

5.2.10 Adicionar funções na base de dados

No ficheiro *database.py* adicione:

```
def get_user(self, username):
    with dbapi2.connect(self.dbfile) as connection:
        cursor = connection.cursor()
        query = "SELECT USERNAME, PASSWORD FROM USER WHERE USERNAME = ?"
        cursor.execute(query, (username,))
        row = cursor.fetchone()

    if row:
        return {"username": row[0], "password": row[1]}
    return None
```

Código 21: Função para obter dados do utilizador.

5.2.11 Proteger páginas

Para garantir que apenas utilizadores autenticados podem modificar dados, podemos utilizar o decorator `@login_required` antes das views que pretendemos proteger:

```
@login_required
def movie_add_page():
```

Também podemos limitar funcionalidades apenas ao administrador, para isso, adiciona-se o seguinte código dentro da função da view.

```
if not current_user.is_admin:
    abort(401)
```

Controlo de acesso

A autenticação garante que o utilizador está identificado. A autorização garante que o utilizador tem permissões para executar determinada ação.

1. **Desafio:** Garanta que apenas o administrador consiga:

- (a) Editar filmes.
- (b) Adicionar filmes.
- (c) Eliminar filmes.

5.3 Desafio extra opcional

Até agora apenas o administrador definido no ficheiro de configuração e os utilizadores existentes na base de dados conseguem fazer *login*.

Como desafio adicional, vamos permitir que novos utilizadores possam ser criados diretamente através da aplicação.

O objetivo é criar um **formulário de registo** que permita adicionar novos utilizadores à base de dados.

1. Comece por criar uma nova página chamada `register.html` na pasta `templates`.

Esta página deve conter um formulário com pelo menos os seguintes campos:

- **Username**
- **Password**
- Botão para criar utilizador

2. No ficheiro `views.py`, implemente uma nova função responsável por mostrar o formulário e processar os dados enviados pelo utilizador.

- Se o pedido for **GET**, deve apresentar o formulário.
- Se o pedido for **POST**, deve validar os dados e criar o utilizador.

3. Antes de guardar a palavra-passe na base de dados, deve aplicar **hashing** utilizando a biblioteca `passlib`.

4. No ficheiro `database.py`, implemente uma função responsável por inserir um novo utilizador na tabela `USER`.

5. Adicione uma nova *route* no ficheiro `server.py` para esta funcionalidade.

6. Adicione um novo botão na página de *login* que permita aceder à página de registo.

Segurança

Ao implementar sistemas de registo de utilizadores é importante garantir que:

- As palavras-passe são sempre armazenadas com *hashing*
- Os nomes de utilizador são únicos
- Os dados recebidos do formulário são devidamente validados

6 Referências

- **Flask Application Development - Basics:** <https://web.itu.edu.tr/uyar/fad/basics.html>
- **Flask Documentation:** <https://flask.palletsprojects.com/>
- **Flask-Login Documentation:** <https://flask-login.readthedocs.io/>
- **Passlib Documentation (Password Hashing):** <https://passlib.readthedocs.io/>
- **Flask-WTF Documentation:** <https://flask-wtf.readthedocs.io/>
- **Web Security Basics – Password Storage Best Practices:** https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- **Flask Application Development – Basics:** <https://web.itu.edu.tr/uyar/fad/basics.html>